```
package lab1;

import static org.junit.Assert.*;

import org.junit.Test;
import java.lang.reflect.Field;

import org.junit.rules.Timeout;
import org.junit.runners.MethodSorters;
import org.junit.FixMethodOrder;
import org.junit.Rule;

@FixMethodOrder(MethodSorters.NAME_ASCENDING)
public class Lab1ClassTest {
        @Rule
        public Timeout globalTimeout = Timeout.seconds(1);

        @Test
        public void test_0_0_minutesSeconds() {
                int x=150;
                String result= Lab1Class.minutesSeconds(x);
                String expect = "150 seconds is 2 minutes and 30 seconds";
                String str = String.format("\nTest minutesSeconds fail for (%d)  Returned (
%s ), but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
        }

        @Test
        public void test_0_1_minutesSeconds() {
                int x=500;
                String result= Lab1Class.minutesSeconds(x);
                String expect = "500 seconds is 8 minutes and 20 seconds";
                String str = String.format("\nTest minutesSeconds fail for (%d)  Returned (
%s ), but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
        }
        @Test
        public void test_0_2_minutesSeconds() {
                int x=50;
                String result= Lab1Class.minutesSeconds(x);
                String expect = "50 seconds is 0 minutes and 50 seconds";
                String str = String.format("\nTest minutesSeconds fail for (%d)  Returned (
%s ), but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
        }
        @Test
        public void test_0_3_minutesSeconds() {
                int x=55820;
                String result= Lab1Class.minutesSeconds(x);
                String expect = "55820 seconds is 930 minutes and 20 seconds";
                String str = String.format("\nTest minutesSeconds fail for (%d)  Returned (
%s ), but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
//              assertEquals(expect,result);
        }

        @Test
        public void test_1_0_temperature() {
                int x=100;
                String result= Lab1Class.temperature(x);
```

```
                String expect = "Fahrenheit 100.0 is 37.78 in Celsius";
                String str = String.format("\nTest temperature fail for (%d)  Returned ( %s
), but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
        }
        @Test
        public void test_1_1_temperature() {
                int x=10;
                String result= Lab1Class.temperature(x);
                String expect = "Fahrenheit 10.0 is -12.22 in Celsius";
                String str = String.format("\nTest temperature fail for (%d)  Returned ( %s
), but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
        }
        @Test
        public void test_1_2_temperature() {
                int x=150;
                String result= Lab1Class.temperature(x);
                String expect = "Fahrenheit 150.0 is 65.56 in Celsius";
                String str = String.format("\nTest temperature fail for (%d) Returned ( %s
), but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
        }
        @Test
        public void test_1_3_temperature() {
                int x=50;
                String result= Lab1Class.temperature(x);
                String expect = "Fahrenheit 50.0 is 10.00 in Celsius";
                String str = String.format("\nTest temperature fail for (%d)  Returned ( %s
), but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
//              assertEquals(expect,result);
        }
        @Test
        public void test_2_0_convertDistance() {
                int x=157;
                String result= Lab1Class.convertDistance(x);
                String expect = "157.0cm is 5ft and 1.81in";
                String str = String.format("\nTest convertDistance fail for (%d)  Returned (
%s ), but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
        }

        @Test
        public void test_2_1_convertDistance() {
                int x=57;
                String result= Lab1Class.convertDistance(x);
                String expect = "57.0cm is 1ft and 10.44in";
                String str = String.format("\nTest convertDistance fail for x=%d  Returned (
%s ), but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
        }
        @Test
        public void test_2_2_convertDistance() {
                int x=37;
                String result= Lab1Class.convertDistance(x);
                String expect = "37.0cm is 1ft and 2.57in";
                String str = String.format("\nTest convertDistance fail for (%d)  Returned (
%s ), but correct is ( %s )\n",
```

```
                                x, result, expect);
                assertEquals(str, expect,result);
        }
        @Test
        public void test_2_3_convertDistance() {
                int x=307;
                String result= Lab1Class.convertDistance(x);
                String expect = "307.0cm is 10ft and 0.87in";
                String str = String.format("\nTest convertDistance fail for (%d)  Returned (
%s ), but correct is ( %s )\n",
                                x, result, expect);
//              assertEquals(expect,result);
                assertEquals(str, expect,result);
        }
        @Test
        public void test_3_0_fiveyearbalance() {
                double x=6000.0, y = 4.25;
                String result= Lab1Class.fiveyearbalance(x,y);
                String expect = "The balance was 6000.00 with interest 4.25, after five
years balance is  7388.08";
                String str = String.format("\nTest fiveyearbalance fail for Balance=%.2f and
Interest=%.2f Returned ( %s ), but correct is ( %s )\n",
                                x,y, result, expect);
                assertEquals(str, expect,result);
        }

        @Test
        public void test_3_1_fiveyearbalance() {
                double x=3000.0, y = 3.5;
                String result= Lab1Class.fiveyearbalance(x,y);
                String expect = "The balance was 3000.00 with interest 3.50, after five
years balance is  3563.06";
                String str = String.format("\nTest fiveyearbalance fail for Balance=%.2f and
Interest=%.2f Returned ( %s ), but correct is ( %s )\n",
                                x,y, result, expect);
                assertEquals(str, expect,result);
        }
        @Test
        public void test_3_2_fiveyearbalance() {
                double x=4000.0, y = 2.25;
                String result= Lab1Class.fiveyearbalance(x,y);
                String expect = "The balance was 4000.00 with interest 2.25, after five
years balance is  4470.71";
                String str = String.format("\nTest fiveyearbalance fail for Balance=%.2f and
Interest=%.2f Returned ( %s ), but correct is ( %s )\n",
                                x,y, result, expect);
                assertEquals(str, expect,result);
        }

        @Test
        public void test_3_3_fiveyearbalance() {
                double x=1000.0, y = 2.25;
                String result= Lab1Class.fiveyearbalance(x,y);
                String expect = "The balance was 1000.00 with interest 2.25, after five
years balance is  1117.68";
                String str = String.format("\nTest fiveyearbalance fail for Balance=%.2f and
Interest=%.2f Returned ( %s ), but correct is ( %s )\n",
                                x,y, result, expect);
//              assertEquals(expect,result);
                assertEquals(str, expect,result);
        }

        @Test
        public void test_4_0_ageinyears() {
                int  x=7000;
```

```
                String result= Lab1Class.ageinyears(x);
                String expect = "Age in days 7000, age in years is 19 with remaining days
 65";
                String str = String.format("\nTest ageinyears fail for (%d) Returned ( %s ),
 but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
        }

        @Test
        public void test_4_1_ageinyears() {
                int  x=17000;
                String result= Lab1Class.ageinyears(x);
                String expect = "Age in days 17000, age in years is 46 with remaining days
 210";
                String str = String.format("\nTest ageinyears fail for (%d) Returned ( %s ),
 but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
        }

        @Test
        public void test_4_2_ageinyears() {
                int  x=37000;
                String result= Lab1Class.ageinyears(x);
                String expect = "Age in days 37000, age in years is 101 with remaining days
 135";
                String str = String.format("\nTest ageinyears fail for (%d) Returned ( %s ),
 but correct is ( %s )\n",
                                x, result, expect);
                assertEquals(str, expect,result);
        }

        @Test
        public void test_4_3_ageinyears() {
                int  x=3650;
                String result= Lab1Class.ageinyears(x);
                String expect = "Age in days 3650, age in years is 10 with remaining days
 0";
                String str = String.format("\nTest ageinyears fail for (%d) Returned ( %s ),
 but correct is ( %s )\n",
                                x, result, expect);
 //             assertEquals(expect,result);
                assertEquals(str, expect,result);
        }


        @Test
        public void test_5_0_mul2Diff() {
                int x=1, y=2;
                int result= Lab1Class.mul2Diff(x, y);
                int expect = -4;
                String str = String.format("\nTest mul2Diff fail for x=%d , y=%d. Returned (
 %d ), but correct is ( %d )\n",
                                x,y, result, expect);
                assertEquals(str, expect,result);

        }

        @Test
        public void test_5_1_mul2Diff() {
                int x=4, y=3;
                int result= Lab1Class.mul2Diff(x, y);
                int expect = -2;
                String str = String.format("\nTest mul2Diff fail for x=%d , y=%d. Returned (
```

```
 %d ), but correct is ( %d )\n",
                                x,y, result, expect);
                assertEquals(str, expect,result);

        }
        @Test
        public void test_5_2_mul2Diff() {
                int x=7, y=6;
                int result= Lab1Class.mul2Diff(x, y);
                int expect = 16;
                String str = String.format("\nTest mul2Diff fail for x=%d , y=%d. Returned (
 %d ), but correct is ( %d )\n",
                                x,y, result, expect);
                assertEquals(str, expect,result);

        }


        @Test
        public void test_6_0_diffSquares() {
                int x=1, y=2;
                int result= Lab1Class.diffSquares(x, y);
                int expect = -3;
                String str = String.format("\nTest diffSquares fail for x=%d , y=%d.
 Returned ( %d ), but correct is ( %d )\n",
                                x,y, result, expect);
                assertEquals(str, expect,result);

        }
        @Test
        public void test_6_1_diffSquares() {
                int x=4, y=3;
                int result= Lab1Class.diffSquares(x, y);
                int expect = 7;
                String str = String.format("\nTest diffSquares fail for x=%d , y=%d.
 Returned ( %d ), but correct is ( %d )\n",
                                x,y, result, expect);
                assertEquals(str, expect,result);

        }
        @Test
        public void test_6_2_diffSquares() {
                int x=6, y=7;
                int result= Lab1Class.diffSquares(x, y);
                int expect = -13;
                String str = String.format("\nTest diffSquares fail for x=%d , y=%d.
 Returned ( %d ), but correct is ( %d )\n",
                                x,y, result, expect);
                assertEquals(str, expect,result);

        }

        @Test
        public void test_7_0_computeSquareRoot() {
                int x = 4;
                String result= Lab1Class.computeSquareRoot(x);
                String expect = "The number (4) has square root value (2.00)";
                String str = String.format("\nTest computeSquareRoot fail for (%d)  Returned
 ( %s ), but correct is ( %s )\n",
                         x,       result, expect);
                assertEquals(str, expect,result);

        }
        @Test
        public void test_7_1_computeSquareRoot() {
```

```
                int x = 5;
                String result= Lab1Class.computeSquareRoot(x);
                String expect = "The number (5) has square root value (2.24)";
                String str = String.format("\nTest computeSquareRoot fail for (%d)  Returned
 ( %s ), but correct is ( %s )\n",
                        x,       result, expect);
                assertEquals(str, expect,result);

        }
        @Test
        public void test_7_2_computeSquareRoot() {
                int x = 56;
                String result= Lab1Class.computeSquareRoot(x);
                String expect = "The number (56) has square root value (7.48)";
                String str = String.format("\nTest computeSquareRoot fail for (%d)  Returned
 ( %s ), but correct is ( %s )\n",
                        x,       result, expect);
                assertEquals(str, expect,result);

        }


        @Test
        public void test_8_0_getBMI() {
                int pounds = 75, feet =15,inches=45;

                double  result= Lab1Class.getBMI(pounds, feet, inches);
                double expect = 1.0;
                String str = String.format("\nTest getBMI fail for pounds=%d,
 feet=%d,inches=%d  Returned ( %.4f ), but correct is ( %.4f )\n",
                                pounds,feet ,inches,     result, expect);
                final double THRESHOLD = .05;
                assertTrue(str, Math.abs(expect - result) < THRESHOLD);

        }
        @Test
        public void test_8_1_getBMI() {
                int pounds = 80, feet =13,inches=45;

                double result= Lab1Class.getBMI(pounds, feet, inches);
                double expect = 1.4;
                String str = String.format("\nTest getBMI fail for pounds=%d,
 feet=%d,inches=%d  Returned ( %.4f ), but correct is ( %.4f )\n",
                                pounds,feet ,inches,     result, expect);
                final double THRESHOLD = .05;
                assertTrue(str, Math.abs(expect - result) < THRESHOLD);

        }

        @Test
        public void test_8_2_getBMI() {
                int pounds = 135, feet =7,inches=45;


                double result= Lab1Class.getBMI(pounds, feet, inches);
                double expect = 5.7;
                String str = String.format("\nTest getBMI fail for pounds=%d,
 feet=%d,inches=%d  Returned ( %.4f ), but correct is ( %.4f )\n",
                                pounds,feet ,inches,     result, expect);
                final double THRESHOLD = .05;
                assertTrue(str, Math.abs(expect - result) < THRESHOLD);
        }


        @Test
```

```
	public void test_9_0_numRevolutions() {
		final long[] DEG = { 0, 1, 360, 719, 720, 800, Long.MAX_VALUE };
		final long[] EXP = { 0, 0, 1, 1, 2, 2, 25620477880152155L };
		for (int i = 0; i < DEG.length; i++) {
			long got = Lab1Class.numRevolutions(DEG[i]);
			long exp = EXP[i];
			String err = String.format("numRevolutions(%s) failed", DEG[i]);
			assertEquals(err, exp, got);
		}
	}


	@Test
	public void test_9_1_numRevolutions() {
		final long DEG = 719;
		final long EXP = 1;

			long got = Lab1Class.numRevolutions(DEG);
			long exp = EXP;
			String err = String.format("numRevolutions(%s) failed", DEG);
			assertEquals(err, exp, got);
		}

	@Test
	public void test_9_2_numRevolutions() {
		final long DEG = 525;
		final long EXP = 1;

			long got = Lab1Class.numRevolutions(DEG);
			long exp = EXP;
			String err = String.format("numRevolutions(%s) failed", DEG);
			assertEquals(err, exp, got);
		}


	@Test
	public void test_9_3_numRevolutions() {
		final long DEG = 955;
		final long EXP = 2;

			long got = Lab1Class.numRevolutions(DEG);
			long exp = EXP;
			String err = String.format("numRevolutions(%s) failed", DEG);
			assertEquals(err, exp, got);
		}

	@Test
	public void test_10_0_fixAngle() {
		final long[] DEG = { 0, 1, 359, 360, 700, 720, 725 };
		final int[] EXP = { 0, 1, 359, 0, 340, 0, 5 };
		for (int i = 0; i < DEG.length; i++) {
			int got = (int)Lab1Class.fixAngle(DEG[i]);
			int exp = EXP[i];
			String err = String.format("fixAngle(%s) failed", DEG[i]);
			assertEquals(err, exp, got);
		}
	}

	@Test
	public void test_10_1_fixAngle() {
		final long DEG = 725;
		final int EXP = 5;

			int got = (int)Lab1Class.fixAngle(DEG);
			int exp = EXP;
```

```
                String err = String.format("fixAngle(%s) failed", DEG);
                assertEquals(err, exp, got);
        }


    @Test
    public void test_10_2_fixAngle() {
        final long DEG = 525;
        final int EXP = 165;

                int got = (int)Lab1Class.fixAngle(DEG);
                int exp = EXP;
                String err = String.format("fixAngle(%s) failed", DEG);
                assertEquals(err, exp, got);
        }

    @Test
    public void test_10_3_fixAngle() {
        final long DEG = 940;
        final int EXP = 220;

                int got = (int)Lab1Class.fixAngle(DEG);
                int exp = EXP;
                String err = String.format("fixAngle(%s) failed", DEG);
                assertEquals(err, exp, got);
        }


    @Test
    public void test_11_0_avg() {
        final int A = 0;
        final int B = 1;
        final int C = Integer.MAX_VALUE-150;
        int a = A;
        int b = B;
        int c = C;
        String exp = "Average of (0, 1, 2147483497) is 715827832.67";
        String error = String.format("avg(%d, %d, %d) failed", a, b, c);
        assertEquals(error, exp, Lab1Class.avg(a, b, c));

    }

    @Test
    public void test_11_1_avg() {
        final int A = 0;
        final int B = 150;
        final int C = Integer.MAX_VALUE-150;

        int a = A;
        int b = B;
        int c = C;
        String exp = "Average of (0, 150, 2147483497) is 715827882.33";
        String error = String.format("avg(%d, %d, %d) failed", a, b, c);
        assertEquals(error, exp, Lab1Class.avg(a, b, c));

    }
    @Test
    public void test_11_2_avg() {
        final int A = -100;
        final int B = -103;
        final int C = -101;

        int a = A;
        int b = B;
        int c = C;
```

```
                String exp = "Average of (-100, -103, -101) is -101.33";
                String error = String.format("avg(%d, %d, %d) failed", a, b, c);
                assertEquals(error, exp, Lab1Class.avg(a, b, c));

        }



}
```